



MTIS | LABORA

LABORA

Proyecto Final MTIS

INTEGRANTES:

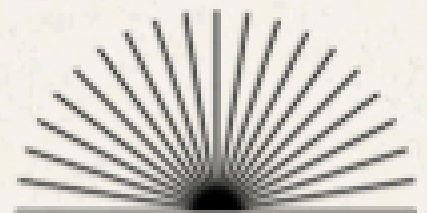
Miriam Amghar Begoug

Hugo Huertas Blasco

Yangfeng Liu

Daniela E Orellana González

Gabriel Manso Gutiérrez de Rozas



1. Escenario de Integración.....	3
2. Definición del Front End y Back End.....	3
2.1 Back End.....	3
2.2 Servicios REST implementados.....	4
2.4 Base de Datos.....	4
2.5 Front End.....	7
3. Puesta en Marcha e Instalación.....	8
3.1 Requisitos previos.....	8
3.2 Instalación de la Base de Datos.....	8
3.3 Arranque de los Servicios SOAP.....	9
3.4 Arranque de los Servicios REST.....	9
4. Problemas Planteados y Soluciones.....	10
5. Conclusiones.....	11
5.1 Sobre la Metodología SOA.....	11
5.2 Sobre REST vs SOAP.....	11
5.3 Sobre las Herramientas Utilizadas.....	11
5.4 ESB - Mulesoft.....	11
5.5 Sobre el Trabajo en Equipo.....	18
5.6 Líneas de Mejora Futuras.....	18

1. Escenario de Integración

El proyecto LABORA consiste en la creación de una plataforma para el Servicio Público de Empleo. El objetivo es interconectar los distintos sistemas implicados en la gestión del empleo y la formación, permitiendo que ciudadanos, empresas y la administración pública interactúen de forma desacoplada y eficiente a través de un conjunto de servicios web.

El sistema integra los siguientes actores y plataformas:

- Portal del Ciudadano: interfaz web para que los demandantes gestionen su perfil, consulten ofertas y cursos, y realicen suscripciones.
- Portal de Empresas: interfaz para que las empresas publiquen ofertas de trabajo y consulten candidatos.
- Sistema de Gestión LABORA (core): núcleo del sistema que gestiona usuarios, empresas, ofertas, cursos, candidaturas, certificados, suscripciones y notificaciones.
- ESB (Enterprise Service Bus): elemento central de integración que enruta, transforma y orquesta las comunicaciones entre todos los servicios.

2. Definición del Front End y Back End

2.1 Back End

El Back End del sistema se compone de dos tipos de servicios web implementados con tecnologías diferentes según la complejidad y naturaleza de cada operación:

Capa	Tecnología	Descripción
Servicios REST	Node.js + Express (openapi-generator-cli)	8 servicios REST generados a partir del contrato OpenAPI unificado. Gestionan las operaciones CRUD sobre los recursos del sistema.
Servicios SOAP	Java + Apache Axis2 + Tomcat 8	3 servicios SOAP implementados a partir de los contratos WSDL. Gestionan operaciones de mayor complejidad transaccional.
Base de Datos	MySQL 5.7 (UniServerZ)	Base de datos relacional con 13 tablas que persiste toda la información del sistema.
ESB	Apache ServiceMix / WSO2	Elemento central de orquestación que coordina los flujos entre servicios.
Servicio de mensajes fakeSMTP	Servidor de fakeSMTP	Elemento que se encarga de recibir las notificaciones de los usuarios

2.2 Servicios REST implementados

Los servicios REST se generaron a partir del contrato OpenAPI unificado mediante openapi-generator-cli con el generador nodejs-express-server. Los 8 servicios desplegados en el puerto 3030 son:

Servicio	Tipo	Endpoints principales
ServicioUsuarios	Entidad	GET/POST /usuarios, GET/PUT/DELETE /usuarios/{id}
ServicioEmpresas	Entidad	GET/POST /empresas, GET/PUT/DELETE /empresas/{cif}
ServicioOfertas	Entidad	GET/POST /ofertas, GET/PUT/DELETE /ofertas/{id}
ServicioCursos	Entidad	GET/POST /cursos, GET/PUT/DELETE /cursos/{id}
ServicioCandidaturas	Entidad	GET/POST /candidaturas, GET/PUT/DELETE /candidaturas/{id}
ServicioSuscripciones	Entidad	GET/POST /suscripciones, GET/DELETE /suscripciones/{id}
ServicioValidacion	Tarea	POST /validacion/usuarios, /validacion/certificados, /validacion/etiquetas, /validacion/elegibilidad
ServicioNotificaciones	Utilitario	POST /notificaciones, GET /notificaciones/{id}

2.4 Base de Datos

La base de datos relacional MySQL se despliega mediante UniServerZ y contiene 13 tablas que cubren todos los servicios del sistema. El esquema incluye las siguientes tablas principales:

- USUARIO: almacena los demandantes, representantes de empresa y administradores.
- EMPRESA: almacena las empresas registradas en el sistema.
- OFERTA: almacena las ofertas de trabajo publicadas por las empresas.
- CURSO: almacena el catálogo de cursos de formación.
- ETIQUETA: catálogo de etiquetas compartido por ofertas, cursos y suscripciones.
- CANDIDATURA: registra las candidaturas de demandantes a ofertas.
- CERTIFICADO: almacena los certificados generados por el ServicioCertificados.
- SUSCRIPCION: registra las preferencias de notificación de los demandantes.
- NOTIFICACION: registro de todas las comunicaciones enviadas por el sistema.
- VALIDACION: registro de las validaciones realizadas por el ServicioValidacion.
- MATCHING_RESULTADO: almacena los resultados del algoritmo de matching SOAP.
- OFERTA_ETIQUETA, CURSO_ETIQUETA, SUSCRIPCION_ETIQUETA: tablas de relación N:M.

Se han definido las tablas en un fichero llamado script y las inserciones están en otro fichero llama insercion:

```

SET FOREIGN_KEY_CHECKS = 0;
DROP DATABASE IF EXISTS labora;
CREATE DATABASE labora CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
USE labora;

CREATE TABLE usuario (
  id_nie VARCHAR(10) NOT NULL COMMENT 'NIF o NIE del ciudadano (PK)',
  nombre VARCHAR(100) NOT NULL,
  apellidos VARCHAR(150) NOT NULL,
  email VARCHAR(254) NOT NULL,
  tipo ENUM(
    'DEMANDANTE',
    'REPRESENTANTE',
    'ADMINISTRADOR'
  ) NOT NULL COMMENT 'Rol del usuario en el sistema',
  activo TINYINT(1) NOT NULL DEFAULT 1 COMMENT '1 = activo, 0 = inactivo',

  -- ServicioIdentidad (WSDL): CrearContraseña / AutenticarUsuario
  contraseña_hash VARCHAR(255) NOT NULL COMMENT 'Hash bcrypt de la contraseña',

  CONSTRAINT pk_usuario PRIMARY KEY (id_nie),
  CONSTRAINT uq_usuario_email UNIQUE (email)
) ENGINE=InnoDB COMMENT='Ciudadanos registrados en la plataforma LABORa';

```

```

USE labora;
SET FOREIGN_KEY_CHECKS = 0;

INSERT INTO usuario (id_nie, nombre, apellidos, email, tipo, activo, contraseña_hash) VALUES
('12345678A', 'María', 'García López', 'maria.garcia@email.com', 'DEMANDANTE', 1, '$2b$12$KIX1234hashejemplo1234abcdefghijklmnop'),
('87654321B', 'Carlos', 'Martínez Pérez', 'carlos.martinez@email.com', 'DEMANDANTE', 1, '$2b$12$KIX1234hashejemplo1234abcdefghijklmnop'),
('11223344C', 'Laura', 'Sánchez Ruiz', 'laura.sanchez@email.com', 'DEMANDANTE', 1, '$2b$12$KIX1234hashejemplo1234abcdefghijklmnop'),
('44332211D', 'Javier', 'López Fernández', 'javier.lopez@email.com', 'DEMANDANTE', 0, '$2b$12$KIX1234hashejemplo1234abcdefghijklmnop'),
('55667788E', 'Ana', 'Romero Castillo', 'ana.romero@email.com', 'DEMANDANTE', 1, '$2b$12$KIX1234hashejemplo1234abcdefghijklmnop'),
('99887766F', 'Pedro', 'Jiménez Moreno', 'pedro.jimenez@email.com', 'DEMANDANTE', 1, '$2b$12$KIX1234hashejemplo1234abcdefghijklmnop'),
('33445566G', 'Elena', 'Torres Navarro', 'elena.torres@email.com', 'DEMANDANTE', 1, '$2b$12$KIX1234hashejemplo1234abcdefghijklmnop'),
('77665544H', 'Roberto', 'Díaz Herrera', 'roberto.diaz@email.com', 'REPRESENTANTE', 1, '$2b$12$KIX1234hashejemplo1234abcdefghijklmnop'),
('22334455I', 'Sofía', 'Molina Vega', 'sofia.molina@email.com', 'REPRESENTANTE', 1, '$2b$12$KIX1234hashejemplo1234abcdefghijklmnop'),
('00112233J', 'Administrador', 'LABORA Sistema', 'admin@labora.gva.es', 'ADMINISTRADOR', 1, '$2b$12$KIX1234hashejemplo1234abcdefghijklmnop');

```

labora

Nueva

candidatura

certificado

curso

curso_etiqueta

empresa

etiqueta

matching_resultado

notificacion

notificacion_destinatario

oferta

oferta_etiqueta

suscripcion

suscripcion_etiqueta

usuario

validacion

Que contengan la palabra:

Tabla	Acción	Filas	Tipo	Cotejamiento	Tamaño	Residuo a depurar
☐ candidatura	★ Examinar Estructura Buscar Insertar Vaciar Eliminar	12	InnoDB	utf8mb4_unicode_ci	64.0 KB	-
☐ certificado	★ Examinar Estructura Buscar Insertar Vaciar Eliminar	7	InnoDB	utf8mb4_unicode_ci	48.0 KB	-
☐ curso	★ Examinar Estructura Buscar Insertar Vaciar Eliminar	10	InnoDB	utf8mb4_unicode_ci	16.0 KB	-
☐ curso_etiqueta	★ Examinar Estructura Buscar Insertar Vaciar Eliminar	25	InnoDB	utf8mb4_unicode_ci	32.0 KB	-
☐ empresa	★ Examinar Estructura Buscar Insertar Vaciar Eliminar	9	InnoDB	utf8mb4_unicode_ci	16.0 KB	-
☐ etiqueta	★ Examinar Estructura Buscar Insertar Vaciar Eliminar	23	InnoDB	utf8mb4_unicode_ci	32.0 KB	-
☐ matching_resultado	★ Examinar Estructura Buscar Insertar Vaciar Eliminar	12	InnoDB	utf8mb4_unicode_ci	64.0 KB	-
☐ notificacion	★ Examinar Estructura Buscar Insertar Vaciar Eliminar	10	InnoDB	utf8mb4_unicode_ci	32.0 KB	-
☐ notificacion_destinatario	★ Examinar Estructura Buscar Insertar Vaciar Eliminar	12	InnoDB	utf8mb4_unicode_ci	32.0 KB	-

Para conectar las APIS con la base de datos se utilizan las siguientes clases:

API Rest: [db.js](#)

```
require('dotenv').config();
const mysql = require('mysql2/promise');

const pool = mysql.createPool({
  host: process.env.DB_HOST || '127.0.0.1',
  user: process.env.DB_USER || 'root',
  password: process.env.DB_PASSWORD || 'root',
  port: Number(process.env.DB_PORT) || 3306,
  database: process.env.DB_NAME || 'labora',
  charset: 'utf8mb4',
  waitForConnections: true,
  connectionLimit: 10,
  queueLimit: 0,
});

module.exports = pool;
```

Este módulo configura y exporta un **connection pool** de MySQL usando **mysql2/promise**, la variante asíncrona del driver que devuelve Promises nativas en lugar de callbacks.

El pool se crea con **mysql.createPool()** leyendo la configuración desde variables de entorno mediante **dotenv** (archivo **.env**), con valores de fallback hardcoded para desarrollo local: host **127.0.0.1**, usuario y contraseña **root**, puerto **3306** y base de datos **labora**. El charset **utf8mb4** garantiza soporte completo de Unicode incluyendo emojis y caracteres especiales.

Los tres parámetros de gestión de conexiones son clave: **waitForConnections: true** hace que las peticiones se encolen si todas las conexiones están ocupadas en lugar de lanzar un error inmediato; **connectionLimit: 10** establece el máximo de conexiones simultáneas abiertas contra MySQL; y **queueLimit: 0** indica que la cola de espera es ilimitada. El pool se exporta como singleton con **module.exports**, de modo que todos los módulos que hagan **require()** de este fichero comparten la misma instancia y el mismo conjunto de conexiones, evitando crear pools duplicados.

API SOAP: Método Connection utilizado en los controladores:

```
private static Connection
getConnection() throws Exception {
    try {
        Class.forName(DRIVER);
        return DriverManager.getConnection(URL, USER, PASSWORD);
    }
    catch (SQLException e) {
        e.printStackTrace();
        throw new Exception("Error interno del servidor");
    }
    catch (ClassNotFoundException e) {
        e.printStackTrace();
        throw new Exception("Error interno del servidor");
    }
}
```

Este método estático privado en Java gestiona la obtención de una conexión individual a la base de datos usando JDBC clásico, a diferencia del pool del ejemplo anterior.

Class.forName(DRIVER) carga dinámicamente el driver JDBC en tiempo de ejecución mediante reflexión — en proyectos modernos con JDBC 4.0+ esto es innecesario ya que el driver se auto-registra, pero se mantiene por compatibilidad. **DriverManager.getConnection(URL, USER, PASSWORD)** abre una conexión nueva cada vez que se invoca, lo que implica que no hay reutilización de conexiones ni pooling, siendo esto un punto débil frente al enfoque de **mysql2/promise**. El método declara **throws Exception** en su firma, obligando a los llamadores a manejar el error.

Los dos bloques **catch** capturan **SQLException** (fallo al conectar, credenciales incorrectas, base de datos inaccesible) y **ClassNotFoundException** (driver no encontrado en el classpath), ambos hacen **e.printStackTrace()** para loguear la traza en consola y luego lanzan una **Exception** genérica con un mensaje opaco **"Error interno del servidor"**, ocultando deliberadamente los detalles del fallo al llamador por seguridad.

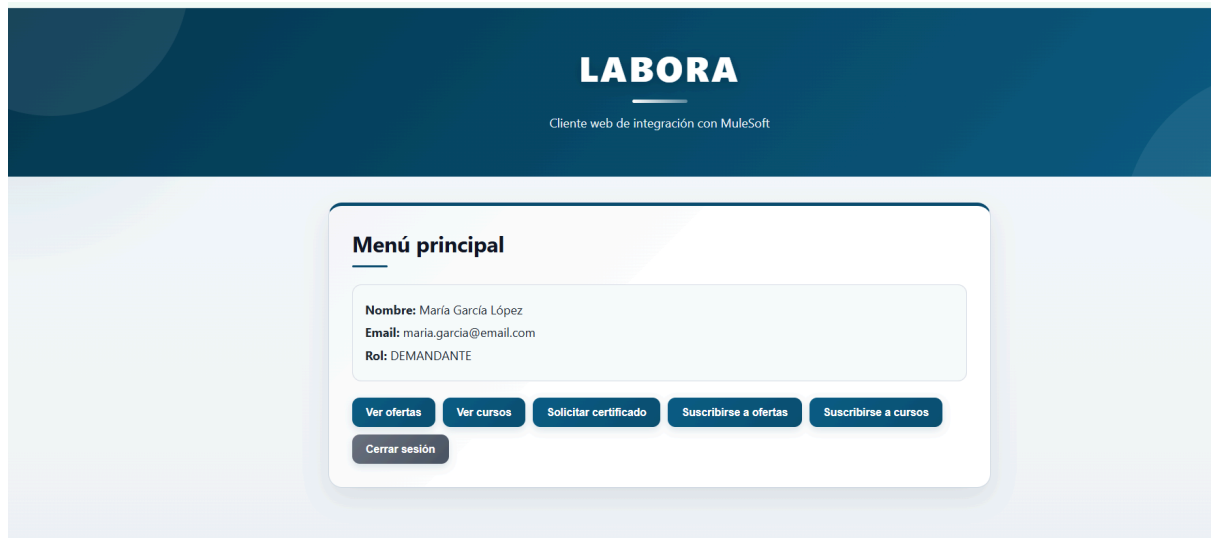
2.5 Front End

El Front End consiste en una aplicación web desarrollada para que los distintos actores del sistema puedan interactuar con los servicios REST y SOAP. La aplicación permite a los ciudadanos registrarse, consultar ofertas y cursos, aplicar a ofertas y gestionar sus suscripciones. Las empresas pueden publicar ofertas y consultar candidatos. Los administradores pueden gestionar certificados y validaciones.

Para el fronted se diseñó un cliente mediante [vite.js](#) y [node.js](#) con html.

En la carpeta fronted (dentro del directorio cliente)

1. Se debe instalar la dependencias: npm install.
2. Se debe ejecutar en la misma carpeta con la instrucción npm run dev.



Se debe tener en marcha los servicios tanto SOAP como Rest, el uniservez y los flows de Mulesoft.

3. Puesta en Marcha e Instalación

A continuación se detallan los pasos necesarios para desplegar y arrancar el sistema de integración completo.

3.1 Requisitos previos

- Java JDK 8 o superior
- Node.js 18 o superior con npm
- Apache Tomcat 8.0
- UniServerZ con MySQL 5.7 activo en el puerto 3307
- Eclipse IDE for Java EE Developers con Apache Axis2
- SoapUI para pruebas de servicios SOAP
- Servidor de fakeSMTP activo en el puerto 2525
- ActiveMQ

3.2 Instalación de la Base de Datos

Con UniServerZ arrancado y MySQL activo, importar los scripts SQL en el siguiente orden desde phpMyAdmin o la consola MySQL:

- Ejecutar el script de creación de tablas: script.mysql
- Ejecutar el script de datos de prueba: labora_inserts.sql

3.3 Arranque de los Servicios SOAP

Los servicios SOAP se despliegan sobre Apache Tomcat 8 desde Eclipse:

- Importar el proyecto labora_soap en Eclipse como proyecto Java EE existente.
- Configurar el servidor Tomcat 8 en Eclipse: Window > Preferences > Server > Runtime Environments.
- Compilar las clases manualmente desde la terminal con javac incluyendo todas las dependencias del directorio WEB-INF/lib.
- Copiar los .class generados a la carpeta WEB-INF/services correspondiente tanto en el proyecto como en el directorio tmp de Eclipse.
- Arrancar Tomcat desde Eclipse. Los servicios estarán disponibles en http://localhost:8080/labora_soap/services.

3.4 Arranque de los Servicios REST

Los servicios REST se arrancan desde la terminal en la carpeta del proyecto Node.js:

- Navegar a la carpeta del proyecto: `cd ServidorLABORA`
- Instalar las dependencias: `npm install --legacy-peer-deps`
- Arrancar el servidor: `npm start` o `node index.js`. El servidor se iniciará en el puerto 3030.

```
C:\Users\mirii\Documents\MTIS-Mat\MTIS-Grupo\mtis_g7\ServidorLABORA>node index.js
Listening on port 3030
{"level":"info","message":"Express server running","service":"user-service","timestamp":"2026-05-18T17:12:27.051Z"}
info: Express server running {"service":"user-service","timestamp":"2026-05-18T17:12:27.051Z"}
█
```

- Verificar el despliegue accediendo a <http://localhost:3030/api-docs> para ver el Swagger UI.

4. Problemas Planteados y Soluciones

Nº	Problema	Solución
1	Conflicto de versiones de slint al instalar dependencias del servidor Node.js generado por openapi-generator-cli.	Ejecutar npm install con el flag --legacy-peer-deps para ignorar los conflictos de versiones entre dependencias de desarrollo.
2	El servidor SOAP devolvía el error 'Please implement' en todos los métodos a pesar de tener el Skeleton implementado.	El problema era que Eclipse no redespiegaba correctamente los .class compilados en Tomcat. La solución fue compilar manualmente con javac todos los .java del paquete a la vez (*.java) y copiar los .class tanto al directorio del proyecto como al directorio tmp de Eclipse.
3	El servicio AutenticarUsuario devolvía siempre 'ok: false' aunque las credenciales eran correctas.	El Controlador.java comparaba la contraseña en texto plano con el hash almacenado en la BD. La solución fue modificar la query para usar SHA2(?, 256) directamente en SQL, delegando el hash a MySQL.
4	El puerto 8080 ya estaba en uso al intentar arrancar el servidor REST, causando el error EADDRINUSE.	Cambiar el puerto del servidor REST al 3030 modificando la variable serverPort en el fichero index.js del proyecto Node.js generado.
5	Error de conexión a la base de datos en el Controlador SOAP: el puerto configurado era el 3306 pero UniServerZ usa el 3307.	Actualizar la cadena de conexión JDBC en Controlador.java: jdbc:mysql://127.0.0.1:3307/labora.
6	Typo en la query de crearContrasena: 'contasena_hash' en lugar de 'contrasena_hash', causando que el UPDATE no afectara a ninguna fila.	Corregir el nombre del campo en la query SQL del método crearContrasena del Controlador.java.
7	El contrato OpenAPI inicial no era coherente con el análisis SOA: faltaban los servicios de Validacion y Suscripciones, el Matching estaba definido como REST cuando debía ser SOAP, y el schema de Candidatura tenía campos incorrectos.	Revisión completa del contrato OpenAPI unificado: eliminación del endpoint /matching, adición de los endpoints /validacion/* y /suscripciones, y corrección del schema CandidaturaInput sustituyendo fecha_publicacion por estado y fecha_aplicacion.

5. Conclusiones

El desarrollo del proyecto de integración LABORA ha permitido aplicar de forma práctica los principios y tecnologías de una Arquitectura Orientada a Servicios (SOA) en un escenario realista y complejo. A continuación se recogen las principales conclusiones obtenidas a lo largo del proyecto.

5.1 Sobre la Metodología SOA

La aplicación de la metodología de Thomas Erl ha resultado fundamental para estructurar el proceso de análisis y diseño. Los 8 pasos aplicados a cada flujo han permitido identificar de forma sistemática los servicios candidatos, clasificarlos correctamente (Entidad, Tarea y Utilitarios) y definir las composiciones entre capas. La distinción entre lógica de negocio en servicios de entidad y lógica de orquestación en el ESB ha sido clave para mantener el desacoplamiento del sistema.

5.2 Sobre REST vs SOAP

La coexistencia de servicios REST y SOAP en el mismo sistema ha demostrado que ambos protocolos tienen su lugar según el tipo de operación. Los servicios REST han resultado más ágiles para las operaciones CRUD sobre recursos, mientras que SOAP ha sido la elección correcta para operaciones de mayor complejidad transaccional como la autenticación de identidad, la generación de certificados oficiales y el algoritmo de matching. La justificación de cada elección ha sido valorada positivamente en el diseño de la arquitectura.

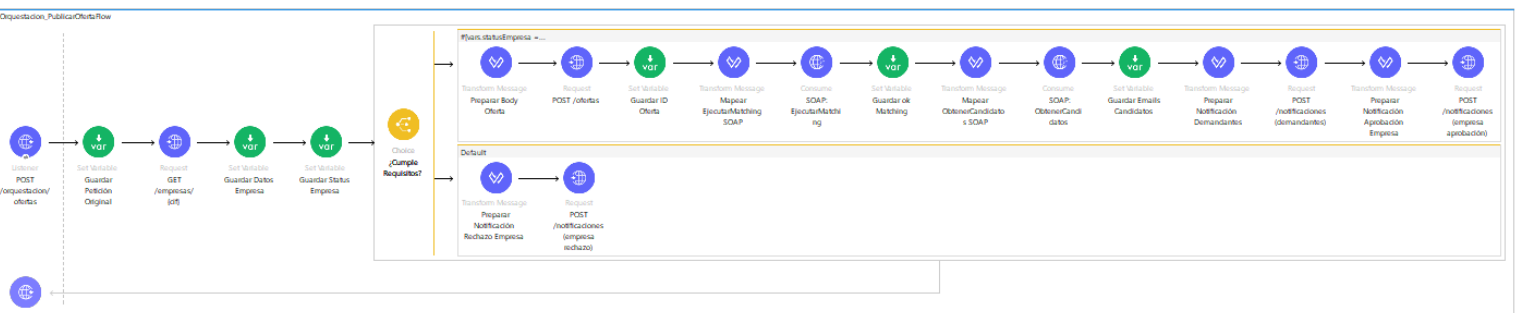
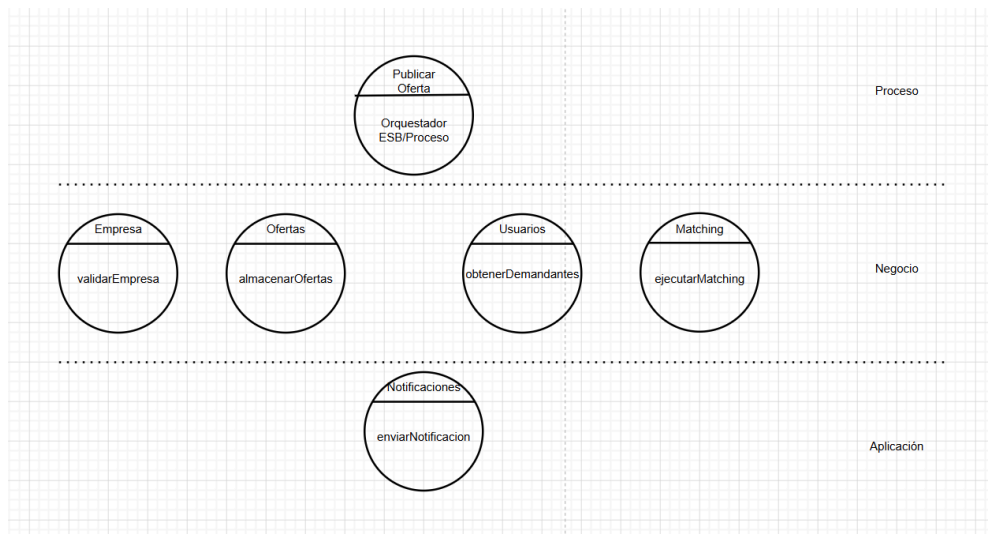
5.3 Sobre las Herramientas Utilizadas

El uso de openapi-generator-cli ha agilizado enormemente la generación del esqueleto de los servicios REST a partir del contrato OpenAPI, evitando errores manuales y garantizando la coherencia entre el contrato y la implementación. Por otro lado, Apache Axis2 con Tomcat ha presentado mayor complejidad en el despliegue, especialmente en la gestión manual de los .class compilados, lo que ha requerido un proceso de compilación y copia manual más laborioso pero que ha permitido comprender en profundidad el ciclo de vida de un servicio SOAP.

5.4 ESB - Mulesoft

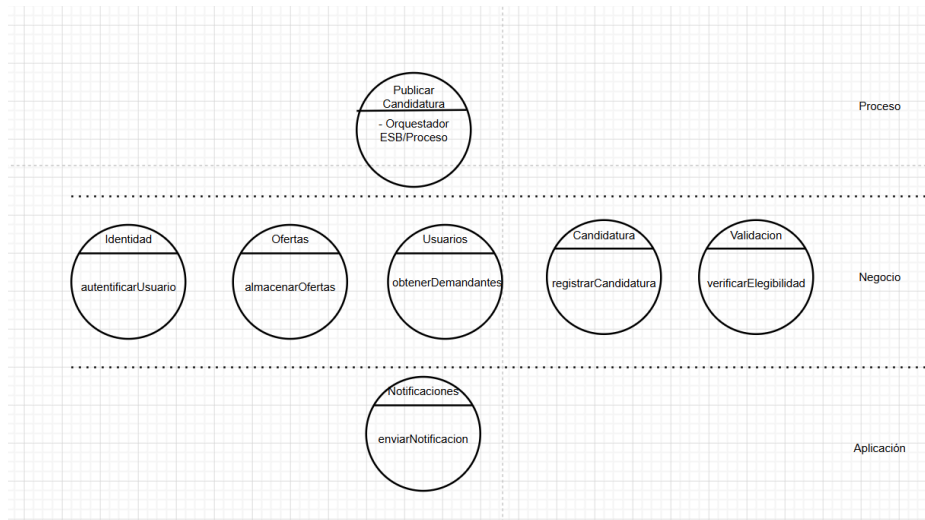
El ESB que se utilizó para orquestar los servicios SOAP y Rest es MuleSoft. En esta aplicación se llama a los servicios mediante condiciones if-else siguiendo los diagramas de flujo del análisis SOA.

1. Publicar oferta

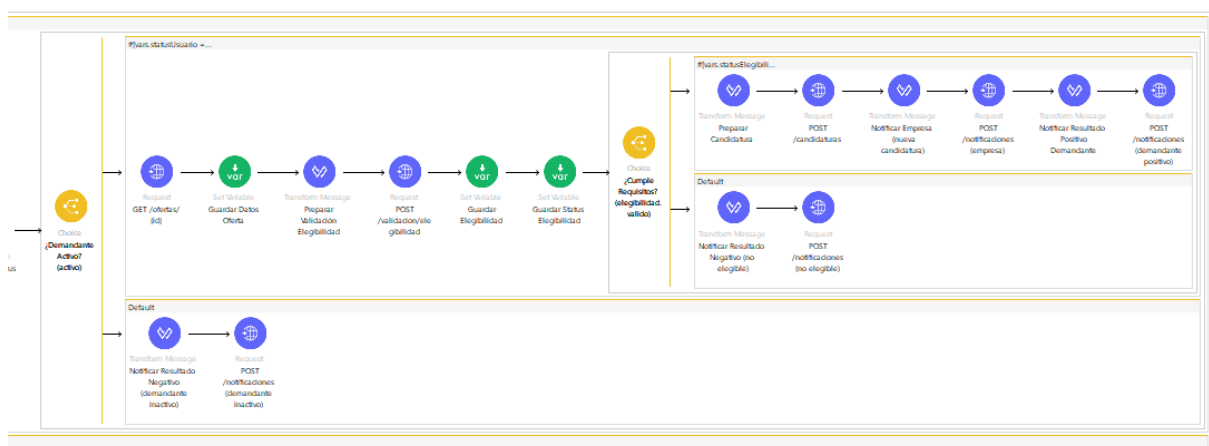
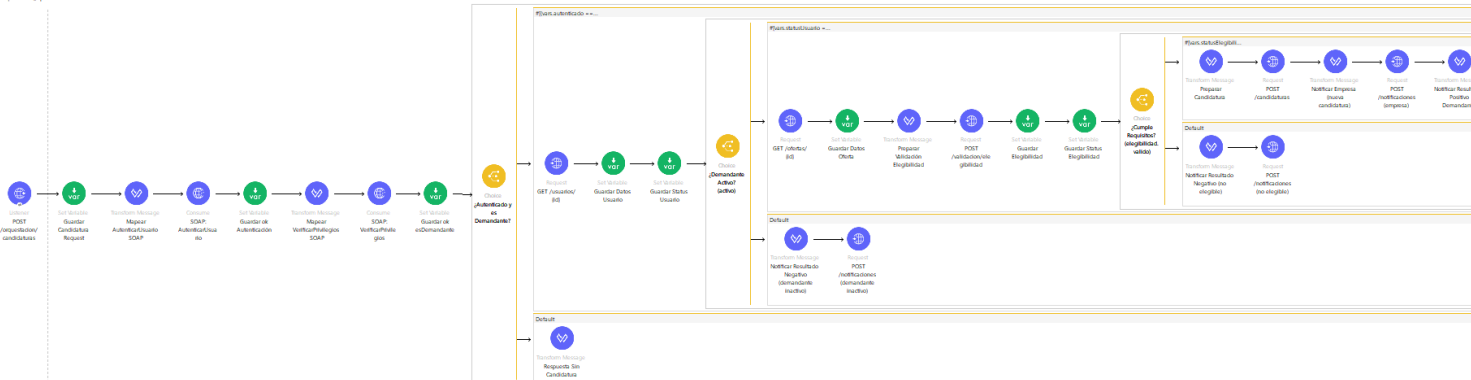


HTTP escucha en **POST /orquestacion/ofertas** (puerto 8082) y almacena el payload en la variable **ofertaOriginal**. Realiza un **GET /empresas/{cif}** contra la API REST (puerto 3030) aceptando 200 y 404 como respuestas válidas, guardando el status code para evaluarlo en un bloque **<choice>**. Si la empresa tiene **activo == true**, ejecuta un **POST /ofertas** para crear la oferta y extrae el **id** resultante (formato "OFE-..."). Con ese id lanza dos operaciones SOAP consecutivas contra **WSC_ServicioMatching**: **EjecutarMatching** y **ObtenerCandidatos**, de cuya respuesta extrae el array de emails con DataWeave usando una comprobación de tipo (**is Array**) para manejar tanto listas como elemento único. Finalmente realiza dos **POST /notificaciones**, uno para los demandantes y otro para la empresa. En el ramal **<otherwise>** notifica el rechazo usando **vars.ofertaOriginal.correo_empresa**.

2. Aplicar oferta.

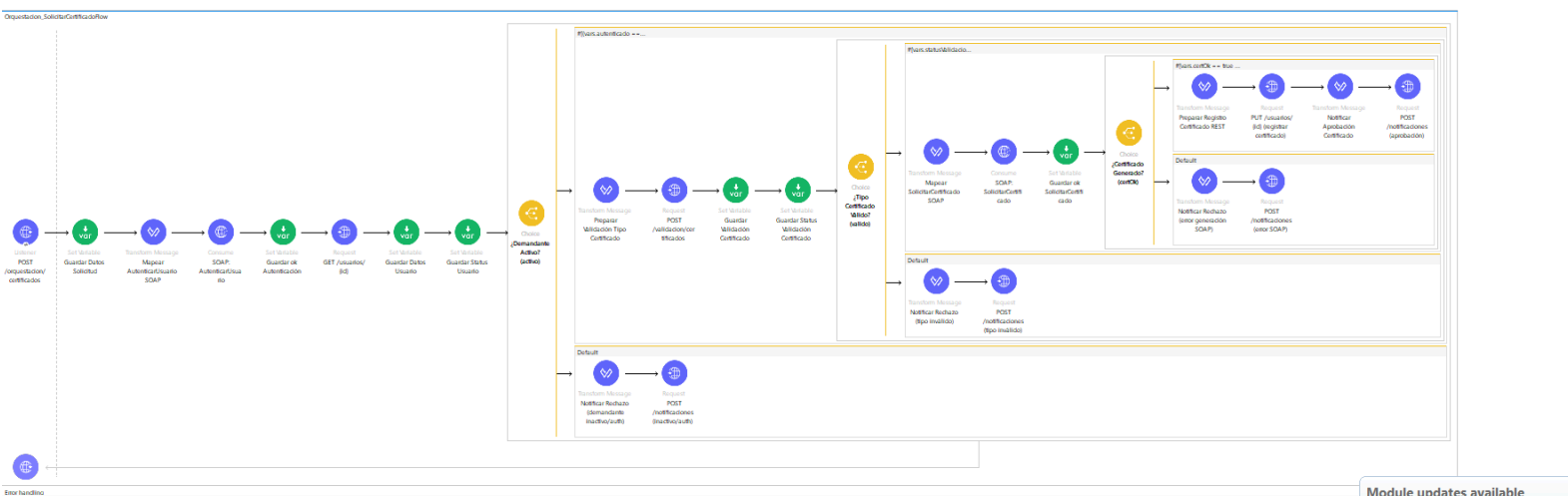
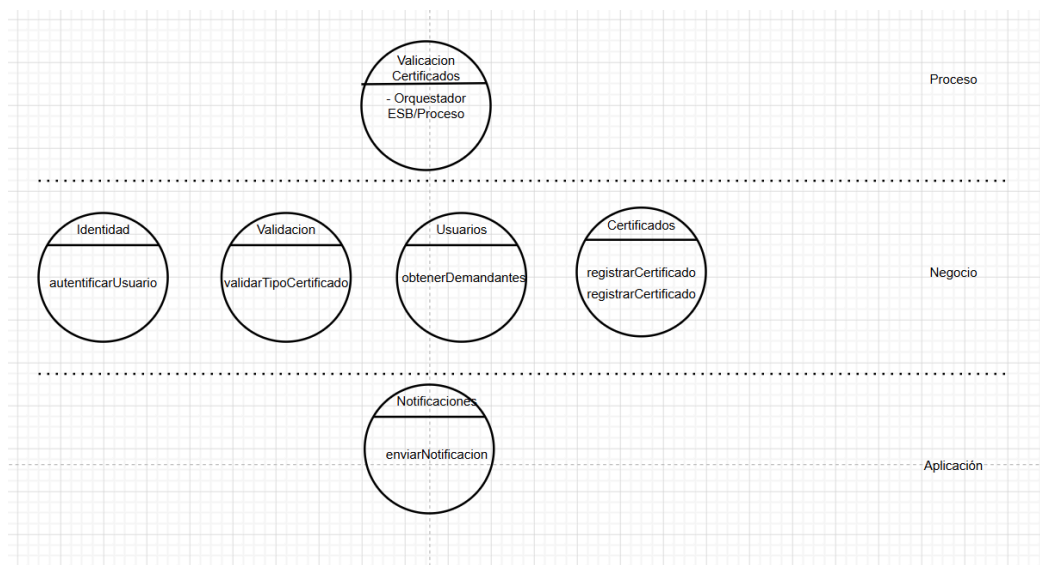


Organization, Application Flow



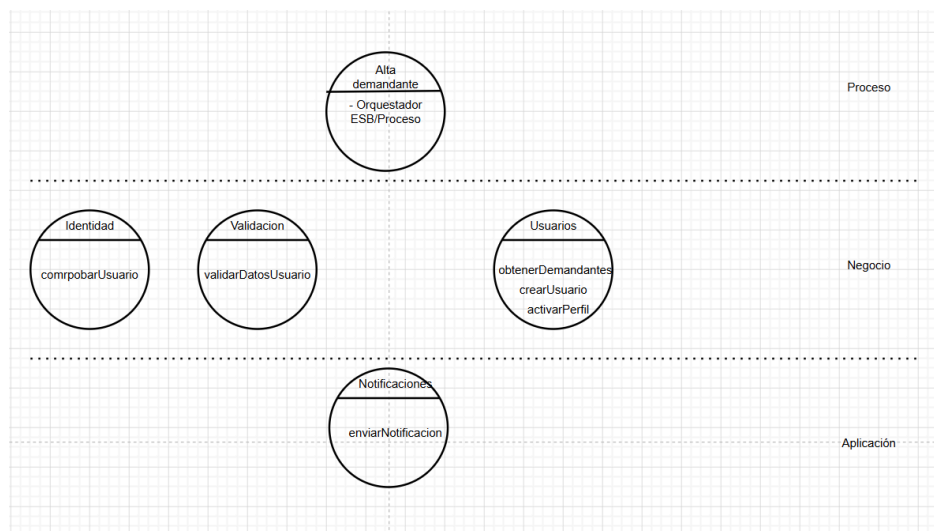
Flujo 2 – Aplicar a una Oferta (**Orquestacion_AplicarOfertaFlow**): El listener atiende **POST /orquestacion/candidaturas** y guarda el payload en **candidaturaReq**. Ejecuta en secuencia dos llamadas SOAP contra **WSC_ServicioIdentidad**: **AutenticarUsuario** (con **nifnie** y **contrasena**) y **VerificarPrivilegios** (con nivel "**DEMANDANTE**"), guardando ambos booleanos **ok** en variables. El primer **<choice>** evalúa ambas condiciones con tolerancia de tipo (**== true or == 'true'**). Si se supera, hace **GET /usuarios/{id}** y anida un segundo **<choice>** que comprueba **activo**. Dentro, un tercer **<choice>** evalúa la respuesta del **POST /validacion/elegibilidad** (acepta 200, 400 y 404); si **valido == true** registra la candidatura con **POST /candidaturas** en estado "**PENDIENTE**" y lanza dos **POST /notificaciones** paralelos a empresa y demandante.

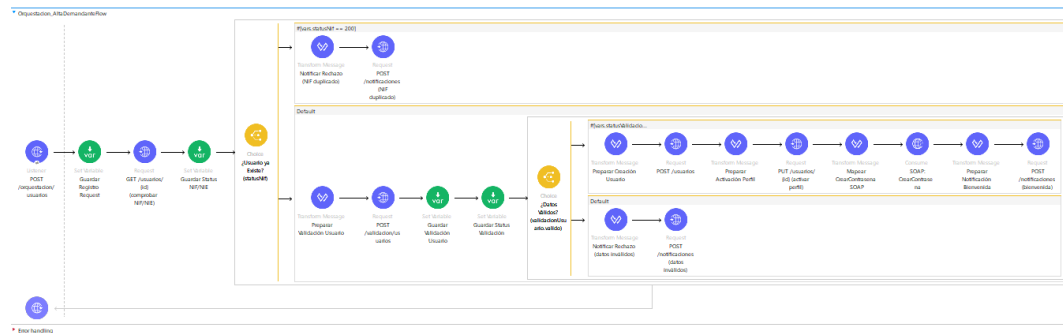
3.Solicitar certificados



Flujo 3 – Solicitar Certificado (**Orquestacion_SolicitarCertificadoFlow**): Tras recibir **POST /orquestacion/certificados**, autentica al usuario con **AutenticarUsuario** SOAP y obtiene sus datos con **GET /usuarios/{id}**. El **<choice>** principal combina en una sola expresión la verificación del booleano **autenticado**, el **statusUsuario == 200** y el campo **activo**. Si se cumplen, valida el tipo de certificado con **POST /validacion/certificados** (acepta 200, 400 y 404). En caso de tipo válido, invoca **SolicitarCertificado** en **WSC_ServicioCertificados** y según el valor de **certOk** ejecuta un **PUT /usuarios/{id}** —con los datos completos del usuario— para registrar el certificado, seguido de un **POST /notificaciones** de aprobación. Hay tres ramales **<otherwise>** anidados que cubren: error SOAP, tipo inválido, y demandante inactivo/no autenticado.

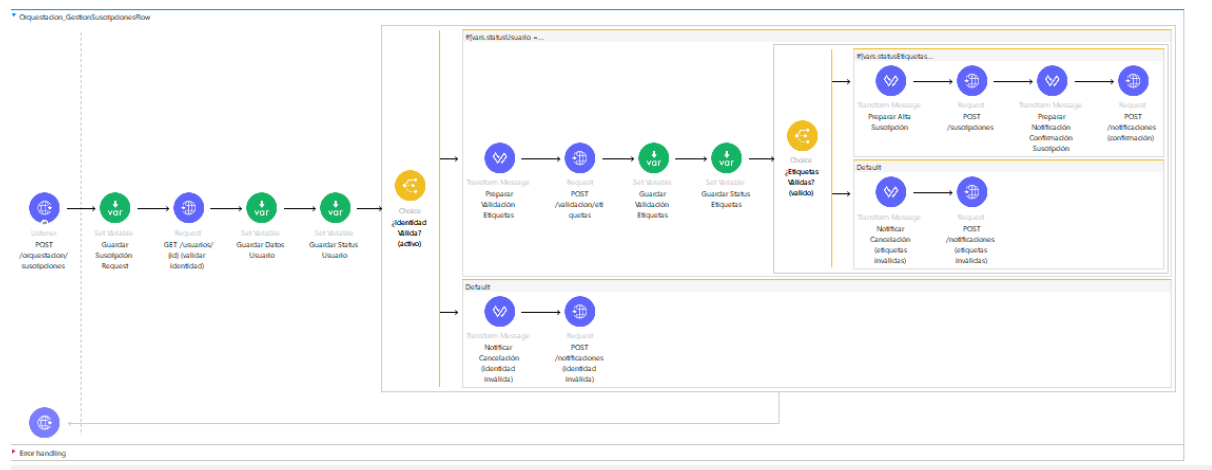
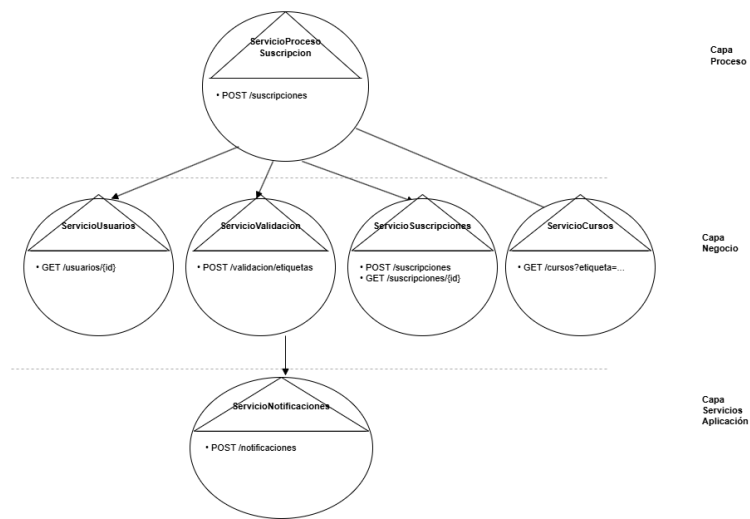
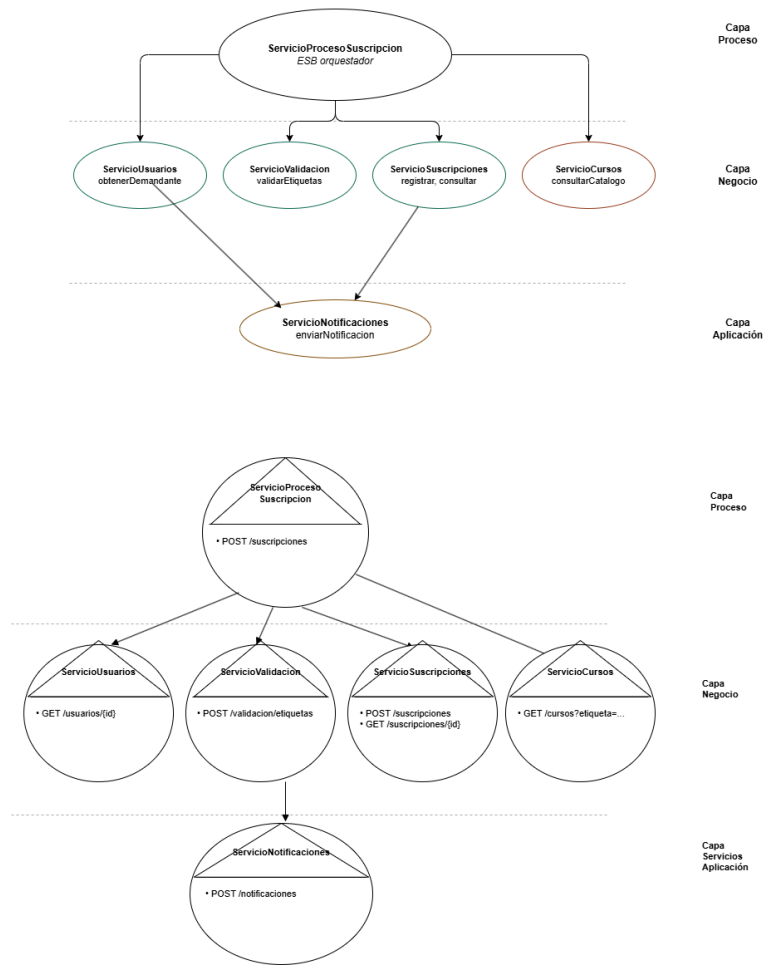
4. AltaDemanda





Flujo 4 – Alta de Demandante (**Orquestacion_AltaDemandanteFlow**): Recibe **POST /orquestacion/usuarios** y comprueba la existencia previa del NIF/NIE con **GET /usuarios/{id}**, evaluando únicamente el **statusCode** (200 = duplicado, 404 = libre). Si el NIF es nuevo, transforma el payload y llama a **POST /validacion/usuarios** con los campos **id_nie**, **nombre**, **apellidos** y **email**. Si **valido == true**, crea el usuario con **POST /usuarios** con **activo: false** y **tipo: "DEMANDANTE"**, lo activa inmediatamente con **PUT /usuarios/{id}** (solo **activo: true** y **tipo**), y establece credenciales llamando a **CrearContraseña** en **WSC_ServicioIdentidad** pasando **nifnie** y **password_inicial**. Finalmente envía la notificación de bienvenida con **tipo: "ALERTA"**.

5. Gestión Suscripción



Flujo 5 – Gestión de Suscripciones (Orquestacion_GestionSuscripcionesFlow):

El listener atiende **POST /orquestacion/suscripciones** y valida la identidad del usuario con **GET /usuarios/{id}**, comprobando **statusUsuario == 200** y **activo**. A continuación ejecuta **POST /validacion/etiquetas** pasando únicamente el array **etiquetas** del request, y almacena tanto el payload como el **statusCode** en variables separadas. Si **valido == true**, construye el body de la suscripción con **id_usuario**, **tipo** y **etiquetas** y lo registra con **POST /suscripciones** (201). La notificación de confirmación usa el operador DataWeave **joinBy "**, **"** para serializar el array de etiquetas en el mensaje. Los dos **<otherwise>** cubren etiquetas inválidas —usando **vars.datosUsuario.email** ya disponible— e identidad inválida —recurriendo a **vars.suscripcionReq.email_usuario** del payload original al no disponer de datos del usuario.

5.5 Sobre el Trabajo en Equipo

El proyecto ha puesto de manifiesto la importancia de la coherencia entre las distintas partes del sistema. La necesidad de alinear el análisis SOA, los contratos OpenAPI y WSDL, la base de datos y la implementación ha requerido una comunicación constante entre los miembros del equipo. Las inconsistencias detectadas en fases avanzadas, como nombres de servicios distintos entre el análisis y los contratos, han evidenciado la importancia de definir y validar los contratos antes de comenzar la implementación.

5.6 Líneas de Mejora Futuras

- Implementar la lógica de negocio completa en los servicios REST, conectando todos los endpoints a la base de datos MySQL.
- Completar la orquestación de los flujos en el ESB, incluyendo las transformaciones de mensajes entre servicios REST y SOAP.
- Añadir autenticación mediante tokens JWT en los servicios REST para complementar el ServicioIdentidad SOAP.
- Desplegar el sistema en un entorno de producción con servidores separados para cada capa, demostrando la escalabilidad de la arquitectura SOA.
- Implementar el paradigma MOM (Message-Oriented Middleware) para las notificaciones asíncronas, mejorando el desacoplamiento del ServicioNotificaciones.